

KAFKA ADVANCE TOPICS

ROUND ROBIN:--

1 Producer → Partition Assignment (Default Behavior)

When a producer sends messages to a **topic with multiple partitions** and **no key is provided**, Kafka distributes messages using **Round Robin** across partitions.

Example:

Topic: `orders`
Partitions: **P0, P1, P2**

Messages sent:

```
msg1 → P0
msg2 → P1
msg3 → P2
msg4 → P0
msg5 → P1
msg6 → P2
kafkaTemplate.send("orders", "message1");
```

when you **provide a key while sending a message**, Kafka **does NOT use round robin** anymore.
Instead, it uses a **hash of the key** to decide the partition.

$\text{partition} = \text{hash}(\text{key}) \% \text{number_of_partitions}$

1 What happens when you provide a key?

Example:

Topic: `orders`
Partitions: **3 (P0, P1, P2)**

If you send messages with key **"user1"**

```
hash("user1") % 3 = P1
```

All messages with **key = user1** will go to **Partition 1**.

Example:

```
key=user1 → P1  
key=user1 → P1  
key=user1 → P1
```

But another key:

```
key=user2 → P0  
key=user3 → P2
```

So Kafka guarantees:

- ✓ **Order is maintained for the same key**
- ✗ **Order is not guaranteed across different keys**

```
kafkaTemplate.send("orders", "order123", "Order created");
```

```
send(topic, key, value)
```

2 Consumer Group Partition Assignment

When multiple consumers belong to the **same consumer group**, Kafka distributes partitions among them.

Some assignor strategies use **round-robin style allocation**.

Example:

Topic partitions:

```
P0, P1, P2, P3
```

Consumers:

```
C1, C2
```

Round robin assignment:

```
C1 → P0, P2  
C2 → P1, P3
```

WAYS TO CREATE KAFKA TOPIS IN SPRINGBOOT

In Apache Kafka with Spring Boot, you can create **Kafka topics mainly in 3 common ways**. In each method you can also specify **number of partitions**.

1 Creating Topic using Spring Boot Bean (Most Common in Projects)

You create a **NewTopic bean**.

Example:

```
import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class KafkaTopicConfig {

    @Bean
    public NewTopic orderTopic() {
        return new NewTopic("orders", 3, (short) 1);
    }
}
```

Explanation:

```
new NewTopic(topicName, partitions, replicationFactor)
```

2 Creating Topic using Kafka CLI (Outside Spring Boot)

Using Kafka command line.

Example:

```
kafka-topics.sh --create \
--topic orders \
--bootstrap-server localhost:9092 \
--partitions 3 \
--replication-factor 1
```

This is usually done by **DevOps or Kafka admins**.

3 Auto Topic Creation (Kafka Broker Feature)

Kafka can automatically create topics when the producer sends a message.

Example:

```
kafkaTemplate.send("orders", "message");
```

If the topic **does not exist**, Kafka will create it automatically.

But:

Partitions will be **default** (usually 1).

Configuration in Kafka broker:

```
auto.create.topics.enable=true
```

⚠ In production, this is usually **disabled**.

How to Mention Partitions in Spring Boot

You specify partitions in **NewTopic**.

Example:

```
@Bean
public NewTopic paymentTopic() {
    return TopicBuilder.name("payment-topic")
        .partitions(4)
        .replicas(1)
        .build();
}
```

This uses **TopicBuilder** (cleaner approach).

Result:

```
Topic → payment-topic
Partitions → 4
Replication → 1
```

How to Send Message to Specific Partition

You can also manually choose a partition.

Example:

```
kafkaTemplate.send("orders", 1, "order123", "Order created");
```

Format:

```
send(topic, partition, key, value)
```

Example result:

```
Topic: orders  
Partition: 1  
Key: order123  
Value: Order created
```

1 What is an Offset?

In Kafka, every message inside a **partition** gets a **unique sequential number** called **offset**.

Example:

```
Topic: orders  
Partition: 0
```

```
Offset 0 → Order created  
Offset 1 → Payment done  
Offset 2 → Order shipped  
Offset 3 → Order delivered
```

So offsets look like:

```
0, 1, 2, 3, 4, 5...
```

Think of **offset like a message index number**.

2 Why Offset is Needed (Very Important)

Offsets help Kafka know:

☞ **Which message a consumer has already processed**

Example:

Consumer reads messages:

```
Offset 0 → processed  
Offset 1 → processed  
Offset 2 → processed  
Offset 3 → next to process
```

Kafka stores:

```
last committed offset = 2
```

So next time the consumer starts, it continues from:

```
Offset 3
```

This prevents **re-reading old messages**.

in Apache Kafka with Spring Boot you **can create multiple listeners for the same topic**. But there are two different ways it behaves, and the **variable you are asking about is usually groupId**.

Let's understand clearly.

1 Multiple Listeners with SAME groupId

Example:

```
@KafkaListener(topics = "orders", groupId = "order-group")  
public void listener1(String message) {  
    System.out.println("Listener1: " + message);  
}  
  
@KafkaListener(topics = "orders", groupId = "order-group")  
public void listener2(String message) {  
    System.out.println("Listener2: " + message);  
}
```

Here:

```
groupId = order-group
```

Both listeners belong to the **same consumer group**.

Result:

Kafka **shares the partitions** between them.

Example:

Topic partitions = 2

Partition0 → listener1
Partition1 → listener2

So **each message is consumed only once.**

2 Multiple Listeners with DIFFERENT groupId

Example:

```
@KafkaListener(topics = "orders", groupId = "group1")
public void listener1(String message) {
    System.out.println("Listener1: " + message);
}

@KafkaListener(topics = "orders", groupId = "group2")
public void listener2(String message) {
    System.out.println("Listener2: " + message);
}
```

Now:

group1
group2

Result:

Each group gets **all messages.**

Example message:

Order Created

Output:

Listener1 → Order Created
Listener2 → Order Created

Because they are **separate consumer groups.**

3 Important Variable in Listener

The key variable is:

groupId

Example:

```
@KafkaListener(  
    topics = "orders",  
    groupId = "order-group"  
)
```

This tells Kafka:

Which consumer group this listener belongs to

4 Another Variable: `id`

You can also give listener id.

Example:

```
@KafkaListener(  
    id = "listener1",  
    topics = "orders",  
    groupId = "order-group"  
)
```

`id` is just **Spring identifier**, not related to Kafka partitioning.

5 Another Important Variable: `concurrency`

Instead of creating multiple listeners, you can create **multiple consumer threads**.

Example:

```
@KafkaListener(  
    topics = "orders",  
    groupId = "order-group",  
    concurrency = "3"  
)
```

Now Spring creates:

3 consumers

Processing partitions in parallel.

🔑 Simple Rule

Case	Result
Same topic + same groupId	Messages shared
Same topic + different groupId	Messages duplicated
concurrency	Multiple consumers in same listener

❑ Very Important Kafka Rule

`Number of active consumers ≤ number of partitions`

Example:

`Partitions = 3`
`Consumers = 5`

Result:

`3 consumers active`
`2 consumers idle`

In Apache Kafka, **Acknowledgement (ACK)** means **confirming that a message has been successfully processed by the consumer**.

When the consumer sends an acknowledgement, Kafka **commits the offset** of that message.

1 Why Acknowledgement is Needed

Kafka needs to know:

`Which messages are already processed`
`Which messages are not processed yet`

Example:

`Topic orders`
`Partition 0`

`Offset 0 → Order Created`
`Offset 1 → Payment Done`
`Offset 2 → Order Shipped`

Consumer processes:

```
Offset 0 → done
Offset 1 → done
Offset 2 → processing
```

When the consumer sends **acknowledgement**, Kafka stores:

```
Last committed offset = 1
```

Next time the consumer starts, it reads from:

```
Offset 2
```

So **acknowledgement** helps Kafka resume correctly.

2 Types of Acknowledgement in Kafka

Type	Description
Auto acknowledgement	Kafka commits offsets automatically
Manual acknowledgement	Developer commits offsets manually

Most production systems prefer **manual acknowledgement**.

3 Enable Manual Acknowledgement in Spring Boot

In Spring Boot first disable auto commit.

```
application.yml
spring:
  kafka:
    consumer:
      enable-auto-commit: false
```

4 Configure Acknowledgement Mode

Create Kafka listener container factory.

```
@Bean
public ConcurrentKafkaListenerContainerFactory<String, String>
kafkaListenerContainerFactory()
```

```

        ConsumerFactory<String, String> consumerFactory) {

    ConcurrentKafkaListenerContainerFactory<String, String> factory =
        new ConcurrentKafkaListenerContainerFactory<>();

    factory.setConsumerFactory(consumerFactory);

    factory.getContainerProperties()
        .setAckMode(ContainerProperties.AckMode.MANUAL);

    return factory;
}

```

Ack modes available:

```

RECORD
BATCH
MANUAL
MANUAL_IMMEDIATE

```

Most used:

```

MANUAL

```

5 Implement Acknowledgement in Listener

Example:

```

@KafkaListener(topics = "orders", groupId = "order-group")
public void consume(String message, Acknowledgment ack) {

    System.out.println("Received: " + message);

    // business logic

    ack.acknowledge();    // commit offset
}

```

Here:

```

Acknowledgment ack

```

is the object used to **manually confirm message processing**.

6 Real Flow

```

Producer sends message
      ↓
Kafka topic
      ↓

```

```
Consumer reads message
      ↓
Business logic executes
      ↓
ack.acknowledge()
      ↓
Offset committed
```

7 What Happens if You DON'T Acknowledge

Example:

```
Offset 5 → message received
Processing fails
acknowledge() NOT called
```

Result:

```
Offset not committed
```

When consumer restarts:

```
Offset 5 will be read again
```

This prevents **message loss**.

8 Why Companies Prefer Manual ACK

Because if you use **auto commit**:

```
Message received
Offset committed
Processing fails
```

Then the message is **lost forever**.

Manual ack ensures:

```
Process message → then commit offset
```

☐ Interview Definition

If asked:

"What is acknowledgement in Kafka?"

Answer:

Acknowledgement in Kafka is the mechanism by which a consumer confirms successful processing of a message so that Kafka can commit the corresponding offset and avoid reprocessing the same message

Retry and DLQ

In real production systems using Apache Kafka with Spring Boot, **Retry and Dead Letter Queue (DLQ)** are used to handle **failed message processing**.

1 Why Retry and DLQ are Needed

Imagine a consumer processing a message:

`Order Created → call payment service`

But the payment service is **temporarily down**.

Without retry:

`Message fails → lost`

This is dangerous in systems like:

- payment systems
- order systems
- banking systems

So we implement:

`Retry → try again few times`
`If still fails → send to DLQ`

2 What is Retry

Retry means:

`If message processing fails`
`→ try processing again`

Example retry flow:

```
Attempt 1 → fail
Attempt 2 → fail
Attempt 3 → success
```

So the message is processed successfully without losing it.

3 What is Dead Letter Queue (DLQ)

If the message keeps failing even after retries, it goes to a **Dead Letter Topic**.

Example:

```
Main Topic → orders
Dead Letter Topic → orders-dlt
```

Flow:

```
orders topic
  ↓
consumer processing
  ↓
retry 3 times
  ↓
still failing
  ↓
send to orders-dlt
```

Now developers can later **inspect the failed messages**.

4 Retry + DLQ Flow in Production

```
Producer
  ↓
orders topic
  ↓
consumer
  ↓
process message
  ↓
success → acknowledge
failure → retry
  ↓
retry limit exceeded
  ↓
send to DLQ
```

5 How to Implement Retry + DLQ in Spring Boot

Spring Kafka provides **DefaultErrorHandler**.

Example configuration:

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<String, String>
kafkaTemplate) {

    FixedBackOff backOff = new FixedBackOff(2000L, 3);

    DeadLetterPublishingRecoverer recoverer =
        new DeadLetterPublishingRecoverer(kafkaTemplate);

    return new DefaultErrorHandler(recoverer, backOff);
}
```

Explanation:

2000L → retry interval (2 seconds)
3 → retry attempts

If retries fail → message goes to **DLQ topic**.

Let's create a **complete Spring Boot Kafka example** that demonstrates:

1. **Manual Acknowledgement (ACK)**
2. **Retry on failure**
3. **Dead Letter Queue (DLQ)**

We'll do it step by step, ready for **real production-like usage**.

1 Dependencies (pom.xml)

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka</artifactId>
    </dependency>
</dependencies>
```

2 Kafka Configuration (application.yml)

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-group
      auto-offset-reset: earliest
      enable-auto-commit: false # manual ack
    producer:
      key-serializer:
org.apache.kafka.common.serialization.StringSerializer
      value-serializer:
org.apache.kafka.common.serialization.StringSerializer
    template:
      default-topic: orders
```

3 Topic Configuration

```
import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class KafkaTopicConfig {

    @Bean
    public NewTopic ordersTopic() {
        return TopicBuilder.name("orders")
            .partitions(3)
            .replicas(1)
            .build();
    }

    @Bean
    public NewTopic ordersDltTopic() {
        return TopicBuilder.name("orders-dlt")
            .partitions(3)
            .replicas(1)
            .build();
    }
}
```

4 Kafka Listener Container Factory with Retry & DLQ

```
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.KafkaTemplate;
```



```

import org.springframework.kafka.listener.DefaultErrorHandler;
import
org.springframework.kafka.listener.ConcurrentKafkaListenerContainerFactory;
import org.springframework.kafka.listener.ContainerProperties;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.util.backoff.FixedBackOff;
import
org.springframework.kafka.support.converter.StringJsonMessageConverter;

@Configuration
public class KafkaListenerConfig {

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, String>
kafkaListenerContainerFactory(
        org.springframework.kafka.core.ConsumerFactory<String, String>
consumerFactory,
        KafkaTemplate<String, String> kafkaTemplate) {

        ConcurrentKafkaListenerContainerFactory<String, String> factory =
            new ConcurrentKafkaListenerContainerFactory<>();

        factory.setConsumerFactory(consumerFactory);

        factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.MAN
UAL);

        // Retry configuration: 3 attempts, 2 seconds interval
        FixedBackOff fixedBackOff = new FixedBackOff(2000L, 3);

        // DLQ recoverer
        org.springframework.kafka.listener.DeadLetterPublishingRecoverer
recoverer =
            new
org.springframework.kafka.listener.DeadLetterPublishingRecoverer(kafkaTempl
ate);

        DefaultErrorHandler errorHandler = new
DefaultErrorHandler(recoverer, fixedBackOff);
        factory.setCommonErrorHandler(errorHandler);

        return factory;
    }
}

```

5 Kafka Producer Service

```

import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class OrderProducer {

    private final KafkaTemplate<String, String> kafkaTemplate;

    public OrderProducer(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }
}

```

```
    public void sendOrder(String orderId, String message) {  
        kafkaTemplate.send("orders", orderId, message);  
    }  
}
```

6 Kafka Consumer with Manual ACK

```
import org.springframework.kafka.annotation.KafkaListener;  
import org.springframework.kafka.support.Acknowledgment;  
import org.springframework.stereotype.Service;  
  
@Service  
public class OrderConsumer {  
  
    @KafkaListener(topics = "orders", groupId = "order-group")  
    public void consume(String message, Acknowledgment ack) {  
  
        System.out.println("Received message: " + message);  
  
        try {  
            // Simulate processing failure for "fail" messages  
            if (message.contains("fail")) {  
                throw new RuntimeException("Processing failed");  
            }  
  
            // Business logic succeeds  
            System.out.println("Processed message successfully");  
  
            // Manually commit offset  
            ack.acknowledge();  
  
        } catch (Exception e) {  
            System.out.println("Error processing message: " +  
e.getMessage());  
            // Don't acknowledge → message goes to retry & eventually DLQ  
            throw e;  
        }  
    }  
}
```

7 How it Works

1. Send message:

```
orderProducer.sendOrder("order123", "Order Created");  
orderProducer.sendOrder("order124", "fail"); // this will go to retry/DLQ
```

2. Consumer reads the message:

- If processing succeeds → `ack.acknowledge()` commits offset.
- If processing fails → `DefaultErrorHandler` retries 3 times with 2 sec interval.
- If all retries fail → message is sent to `orders-dlt`.

3. **DLQ topic:**

- Developers can inspect failed messages in `orders-dlt`.
- You can later reprocess them manually.